

Code-Layout, Readability and Reusability

Date	06 July 2026
Project Name	Credit Card Approval Prediction System
Prepared By	Joyal Kumar, Keerthi, Yasawini, Nasir, Harika

This assessment is based on a direct read-through of the actual repository (main.py, app.py, templates/, static/) rather than a generic checklist fill-in - findings and remarks reflect what is genuinely in the code.

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Both main.py and app.py use consistent 4-space indentation throughout.
2	Proper File Structure	Files and folders are logically organized	Partial	templates/ and static/ are correctly separated Flask-style, but data files (CSV), model.pkl, and code sit together in the root with no src/ or data/ subfolder, and a .venv folder is committed to the repo instead of being git-ignored.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names like income, education, family_status, days_birth are self-explanatory. Short names (X, y, lr_model, dt_model, rf_model) are used but are conventional for ML code.
4	Function / Method Names	Functions are descriptively named	Partial	Flask routes (home, predict) are clearly named. However, main.py has no functions at all - it is one long top-to-bottom script rather than being broken into functions like load_data(), preprocess(), train_models().
5	Code Comments	Inline and block comments explain logic	Yes	main.py uses clear section-divider comments (e.g., 'Data Pre-processing', 'Model Building') and inline notes explaining key steps like the Good/Bad target encoding logic.
6	Modular Design	Code is split into reusable functions/modules	No	main.py is a single monolithic script with no functions, classes, or reusable modules - the same preprocessing logic cannot currently be imported/reused by app.py, which is why the LabelEncoder used at inference isn't guaranteed to match training (see Remarks below).
7	No Redundant Code	Duplicate or unused code is removed	Partial	main.py repeats near-identical train/predict/evaluate blocks for all three models (Logistic Regression, Decision Tree, Random Forest) instead of looping over a models dict. Several matplotlib/seaborn plotting blocks are commented out rather than removed.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	app.py wraps /predict in a try/except, but on failure it returns the raw exception text ("Error : {e}") directly to the user instead of a friendly message or proper HTTP error code - acceptable for a class project demo, not production-ready.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	model.pkl (trained Random Forest)	Python / scikit-learn (pickle)	Loaded once by app.py and reused for every /predict request without retraining	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
2	static/style.css	CSS	Shared styling across index.html and result.html	Medium
3	Flask route pattern (home / predict)	Python / Flask	Follows a standard, easily-extensible Flask app pattern that could be reused as a template for similar single-form ML prediction apps	Medium
4	Data preprocessing logic (main.py)	Python / pandas	Not currently reusable - it is inline script code, not a function/module app.py can import, so the same cleaning/encoding steps had to be manually re-implemented in app.py's feature array	Low
5	-	-	-	-

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	3	Clean within each file, but project-level organization is flat (no src/, data/, or config separation) and .venv is committed to version control.
Readability	4	Descriptive names and clear section comments in main.py make the logic easy to follow even without prior context.
Reusability	2	The trained model itself is reusable via pickle, but the preprocessing/encoding logic is not factored out, so it had to be duplicated by hand between main.py and app.py.
Documentation / Comments	3	Good inline comments explain what each block does, but there is no README, docstrings, or requirements.txt describing how to set up and run the project.
Overall Score	3.0	A solid, readable first version of the project. The main improvement opportunity is refactoring main.py into reusable functions (e.g., a shared preprocessing module) and persisting the label encoder alongside the model.